llama.cpp

```
llama-server \
  -m model.gguf \
  -ngl 999 \
  -c 32768 \
  -b 2048 \
  -ub 512 \
  --flash-attn on
```

llama.cpp SERVER

```
llama-server \
  -m model.gguf \
  -ngl 999 \
  -c 32768 \
  -b 2048 \
  -ub 512 \
  --flash-attn on
```

`llama-server`

Starts llama.cpp as a local HTTP server with an OpenAI-compatible API.

llama.cpp MODEL

```
llama-server \
  -m model.gguf \
  -ngl 999 \
  -c 32768 \
  -b 2048 \
  -ub 512 \
  --flash-attn on
```

`-m model.gguf`

Specifies the GGUF model file that llama.cpp should load.

llama.cpp GPU

```
llama-server \
  -m model.gguf \
  -ngl 999 \
  -c 32768 \
  -b 2048 \
  -ub 512 \
  --flash-attn on
```

`-ngl 999`

Sets the maximum number of model layers to store in VRAM. 999 effectively requests full GPU offload.

llama.cpp CONTEXT

```
llama-server \
  -m model.gguf \
  -ngl 999 \
  -c 32768 \
  -b 2048 \
  -ub 512 \
  --flash-attn on
```

`-c 32768`

Sets the context size to 32,768 tokens.

llama.cpp BATCHING

```
llama-server \
  -m model.gguf \
  -ngl 999 \
  -c 32768 \
  -b 2048 \
  -ub 512 \
  --flash-attn on
```

`-b 2048`

Sets the logical maximum batch size used during prompt processing.

llama.cpp BATCHING

```
llama-server \
  -m model.gguf \
  -ngl 999 \
  -c 32768 \
  -b 2048 \
  -ub 512 \
  --flash-attn on
```

`-ub 512`

Sets the maximum physical batch size processed at one time.

llama.cpp ATTENTION

```
llama-server \
  -m model.gguf \
  -ngl 999 \
  -c 32768 \
  -b 2048 \
  -ub 512 \
  --flash-attn on
```

`--flash-attn on`

Forces Flash Attention on, which can improve attention performance and reduce memory usage on supported hardware.

llama.cpp CPU

```
llama-server [core options] \
  -t 6 \
  -tb 12 \
  -ctk q8_0 \
  -ctv q8_0
```

`-t 6`

Sets the number of CPU threads used during token generation.

llama.cpp CPU

```
llama-server [core options] \
  -t 6 \
  -tb 12 \
  -ctk q8_0 \
  -ctv q8_0
```

`-tb 12`

Sets the number of CPU threads used during prompt and batch processing.

llama.cpp KV CACHE

```
llama-server [core options] \
  -t 6 \
  -tb 12 \
  -ctk q8_0 \
  -ctv q8_0
```

`-ctk q8_0`

Stores the K portion of the KV cache using Q8_0 quantization to reduce memory usage.

llama.cpp KV CACHE

```
llama-server [core options] \
  -t 6 \
  -tb 12 \
  -ctk q8_0 \
  -ctv q8_0
```

`-ctv q8_0`

Stores the V portion of the KV cache using Q8_0 quantization to reduce memory usage.

llama.cpp CONCURRENCY

```
llama-server [core options] [performance options] \
  -np 4 \
  --host 0.0.0.0 \
  --port 8080 \
  --api-key YOUR_KEY
```

`-np 4`

Creates four server slots, allowing llama-server to handle multiple requests concurrently.

llama.cpp NETWORK

```
llama-server [core options] [performance options] \
  -np 4 \
  --host 0.0.0.0 \
  --port 8080 \
  --api-key YOUR_KEY
```

`--host 0.0.0.0`

Makes llama-server listen on all network interfaces instead of only localhost.

llama.cpp NETWORK

```
llama-server [core options] [performance options] \
  -np 4 \
  --host 0.0.0.0 \
  --port 8080 \
  --api-key YOUR_KEY
```

`--port 8080`

Sets the TCP port used by llama-server. The default is port 8080.

llama.cpp SECURITY

```
llama-server [core options] [performance options] \
  -np 4 \
  --host 0.0.0.0 \
  --port 8080 \
  --api-key YOUR_KEY
```

`--api-key YOUR_KEY`

Requires API-key authentication. Especially useful when exposing llama-server beyond localhost.

llama.cpp SAMPLING

```
llama-server [core options] [performance options] \
  [server options] \
  --temp 0.8 \
  --top-p 0.95 \
  --min-p 0.05 \
  --repeat-penalty 1.1 \
  --seed 42
```

`--temp 0.8`

Controls randomness in token selection. Lower values are more deterministic; higher values allow more variation.

llama.cpp

SAMPLING

PROMPT

Write a tagline for a local AI testing channel.

`--temp 0.2`

LLAMA

Independent AI testing with clear, practical results.

`--temp 1.2`

LLAMA

Local AI, pushed hard. Real models, real hardware, real results.

Illustrative outputs — exact generations vary.

llama.cpp SAMPLING

```
llama-server [core options] [performance options] \
  [server options] \
  --temp 0.8 \
  --top-p 0.95 \
  --min-p 0.05 \
  --repeat-penalty 1.1 \
  --seed 42
```

`--top-p 0.95`

Limits sampling to the most likely tokens whose combined probability reaches 95%.

llama.cpp

SAMPLING

PROMPT

Write a one-sentence description of a local AI benchmark channel.

`--top-p 0.70`

LLAMA

A channel focused on practical AI testing and benchmark results.

`--top-p 0.95`

LLAMA

A hands-on channel exploring local AI models, real hardware, and benchmark-driven results.

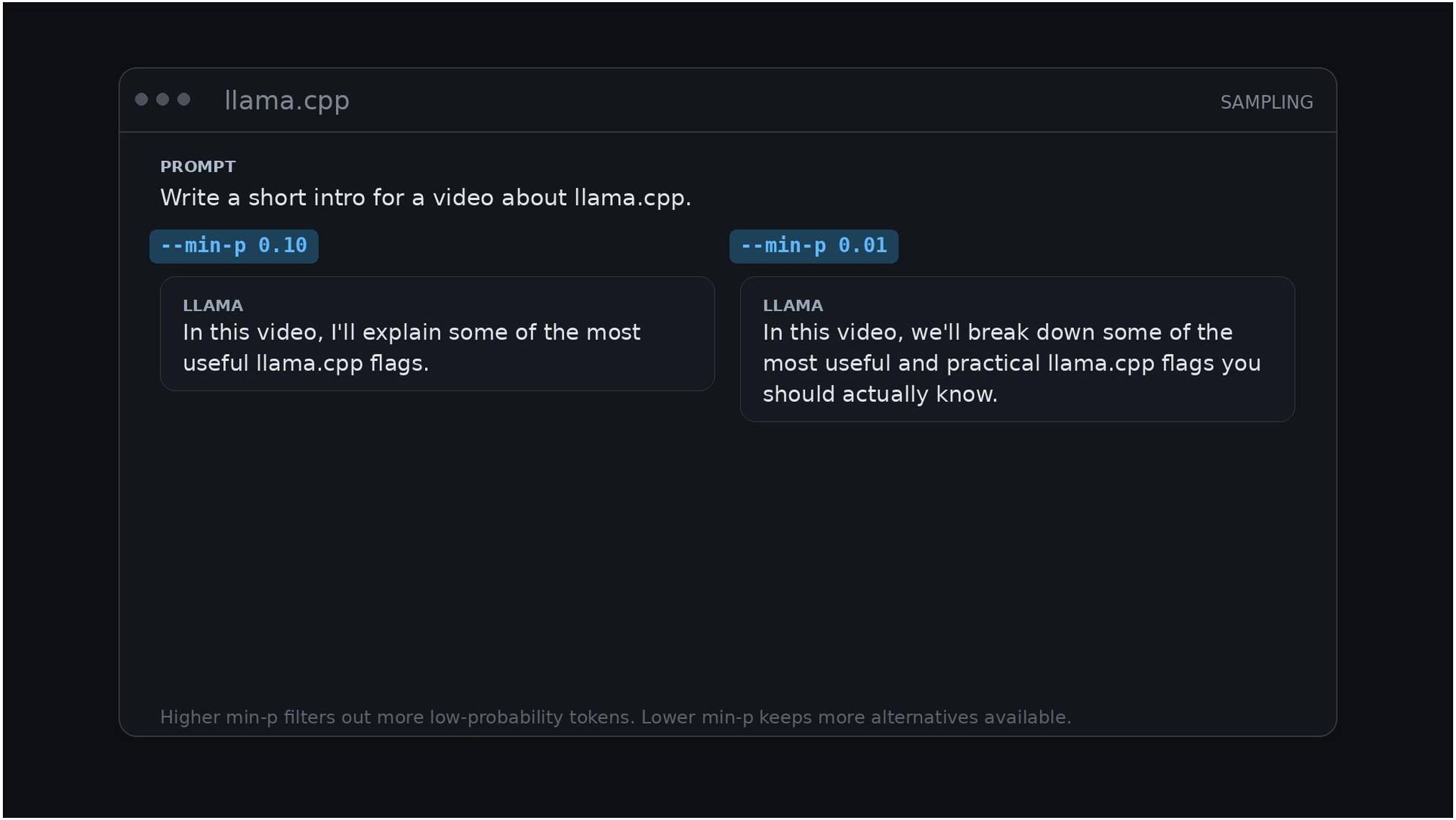
Lower top-p keeps sampling within a tighter probability mass. Higher top-p allows more possible tokens.

llama.cpp SAMPLING

```
llama-server [core options] [performance options] \
  [server options] \
  --temp 0.8 \
  --top-p 0.95 \
  --min-p 0.05 \
  --repeat-penalty 1.1 \
  --seed 42
```

`--min-p 0.05`

Filters out tokens that are too unlikely relative to the model's most probable token.


llama.cpp
SAMPLING
PROMPT
Write a short intro for a video about llama.cpp.
--min-p 0.10
LLAMA
In this video, I'll explain some of the most useful llama.cpp flags.
--min-p 0.01
LLAMA
In this video, we'll break down some of the most useful and practical llama.cpp flags you should actually know.
Higher min-p filters out more low-probability tokens. Lower min-p keeps more alternatives available.

llama.cpp SAMPLING

```
llama-server [core options] [performance options] \
  [server options] \
  --temp 0.8 \
  --top-p 0.95 \
  --min-p 0.05 \
  --repeat-penalty 1.1 \
  --seed 42
```

`--repeat-penalty 1.1`

Penalizes recently used tokens to reduce repetitive words and phrases. A value of 1.0 disables the penalty.

llama.cpp

SAMPLING

PROMPT

List the benefits of local AI inference.

`--repeat-penalty 1.0`

LLAMA

Local inference gives you more privacy, more control, and more flexibility. It also gives you more independence and more control over your workflow.

`--repeat-penalty 1.2`

LLAMA

Local inference gives you more privacy, more control, and more flexibility. It also helps with independence, customization, and predictable access to your own workflow.

A stronger repetition penalty discourages the model from reusing the same words and phrases.

llama.cpp SAMPLING

```
llama-server [core options] [performance options] \
  [server options] \
  --temp 0.8 \
  --top-p 0.95 \
  --min-p 0.05 \
  --repeat-penalty 1.1 \
  --seed 42
```

`--seed 42`

Sets the random seed used during generation. Using the same seed can help make tests more reproducible.

llama.cpp

SAMPLING

PROMPT

Write a short tagline for a llama.cpp cheat sheet video.

`--seed 42`

LLAMA

A practical llama.cpp cheat sheet for real-world local AI use.

`--seed 99`

LLAMA

Your quick guide to the most useful llama.cpp flags and settings.

Different seeds can produce different outputs even with the same prompt and settings.

llama.cpp SAMPLING

```
llama-server [previous options] \
  --top-k 40 \
  --mlock \
  --no-mmap \
  --lora adapter.gguf \
  --mmproj mmproj.gguf
```

`--top-k 40`

Limits token selection to the 40 most likely candidates before sampling.

llama.cpp

SAMPLING

PROMPT

Write a tagline for a local AI testing channel.

`--top-k 10`

LLAMA

Local AI testing with practical, measurable results.

`--top-k 100`

LLAMA

Real models. Real hardware. Local AI pushed beyond the demo.

Lower top-k restricts choices more. Higher top-k allows a wider candidate pool.

llama.cpp MEMORY

```
llama-server [previous options] \
  --top-k 40 \
  --mlock \
  --no-mmap \
  --lora adapter.gguf \
  --mmproj mmproj.gguf
```

`--mlock`

Locks model data in RAM to help prevent the operating system from swapping it to disk.

llama.cpp MEMORY

```
llama-server [previous options] \
  --top-k 40 \
  --mlock \
  --no-mmap \
  --lora adapter.gguf \
  --mmproj mmproj.gguf
```

`--no-mmap`

Disables memory-mapped model loading and loads model data directly into memory instead.

llama.cpp LORA

```
llama-server [previous options] \
  --top-k 40 \
  --mlock \
  --no-mmap \
  --lora adapter.gguf \
  --mmproj mmproj.gguf
```

`--lora adapter.gguf`

Loads a LoRA adapter and applies it on top of the base model.

llama.cpp MULTIMODAL

```
llama-server [previous options] \
  --top-k 40 \
  --mlock \
  --no-mmap \
  --lora adapter.gguf \
  --mmproj mmproj.gguf
```

`--mmproj mmproj.gguf`

Loads the multimodal projector required by compatible vision and multimodal models.

llama.cpp MULTI-GPU

```
llama-server [previous options] \
  --top-k 40 \
  --mlock \
  --no-mmap \
  --lora adapter.gguf \
  --mmproj mmproj.gguf
```

`-sm layer`

Sets how model data is distributed across multiple GPUs. Layer mode splits model layers between them.

llama.cpp MULTI-GPU

```
llama-server [previous options] \
  -sm layer \
  -ts 1,1 \
  -mg 0
```

`-sm layer`

Sets how model data is distributed across multiple GPUs. Layer mode splits model layers between them.

llama.cpp MULTI-GPU

```
llama-server [previous options] \
  --top-k 40 \
  --mlock \
  --no-mmap \
  --lora adapter.gguf \
  --mmproj mmproj.gguf
```

`-ts 1,1`

Sets the relative tensor split between GPUs. A 1,1 split requests an even distribution across two GPUs.

llama.cpp MULTI-GPU

```
llama-server [previous options] \
  -sm layer \
  -ts 1,1 \
  -mg 0
```

`-ts 1,1`

Sets the relative tensor split between GPUs. A 1,1 split requests an even distribution across two GPUs.

llama.cpp MULTI-GPU

```
llama-server [previous options] \
  --top-k 40 \
  --mlock \
  --no-mmap \
  --lora adapter.gguf \
  --mmproj mmproj.gguf
```

`-mg 0`

Selects GPU 0 as the main GPU for operations whose placement depends on the selected split mode.

llama.cpp MULTI-GPU

```
llama-server [previous options] \
  -sm layer \
  -ts 1,1 \
  -mg 0
```

`-mg 0`

Selects GPU 0 as the main GPU for operations whose placement depends on the selected split mode.